STANDARD GUIDE

How an agent registers and pays

Anyone connecting an agent can follow this. It covers the normal PayAgent path and the optional Tether WDK wallet. Date: 27 September 2026.

1. The one thing to remember

Registering an agent only creates a name in PayAgent. It does not create a wallet and it does not move money. The wallet lives on the agent machine. PayAgent never sees the seed or private key.

You can use Tether WDK, Phantom, MetaMask, or any other signer. WDK is one option, not a lock-in. If you use WDK, install the package on the agent first, then attach the address it prints.

2. Who does what

You / portal	Create an API key. See agents, wallets, and policies.
Agent machine	Holds the wallet. Signs and sends the on-chain payment.
PayAgent	Checks the rules, then verifies the on-chain txHash.
WDK (optional)	Opens the wallet and sends the token. Never hosted by us.

3. The path, start to finish

1. Create a merchant API key in the PayAgent portal.
2. If the agent uses WDK: install the package on that machine (see section 6). Open a wallet. Copy the address.
3. Register the agent. You only send a name. No wallet is created here.
4. Attach the wallet address to that agent (Solana, Ethereum, or Base).
5. Set a spend policy (which network, which token, how much, approval).
6. Fund the wallet on-chain. The agent pays its own gas. PayAgent does not sponsor fees.
7. Ask PayAgent “may I pay?” (preflight). If ready, the wallet sends.
8. Report the txHash. PayAgent checks the chain. If it matches, the payment is paid.

4. How registration works

Registration is an API call with your merchant key. It creates an agent row: a public id (agt_…) and the name you chose. After that the agent appears under Registered Agents.

`POST /api/v1/agents/register` body: agent name (and optional your own agent id).

What this step does not do:

- It does not install any package.
- It does not create or fund a wallet.
- It does not let the agent pay yet.

Next you bind a payer address. PayAgent will only accept payments from that address. A different wallet gets AGENT_WALLET_MISMATCH.

`POST /api/v1/agents/wallet` body: agent id, wallet address, network (for example solana).

Then you set the rules. Without an active policy, preflight stays blocked.

`PUT /api/v1/compliance/agents/{agentId}/policy`

In the portal you will see the agent name, id, and wallet. You can copy the wallet from Registered Agents or Wallets ’ Agents. That address is what you fund.

5. How payment works

Money never sits in PayAgent. The agent wallet sends the token on the real chain. PayAgent’s job is: allow or deny, then prove the transfer happened.

1. Preflight — GET /api/v1/pay/preflight. PayAgent checks: agent exists, wallet matches, policy allows this token / network / amount, and no extra human approval is required.

2. If ready is false, stop. Do not send. Fix funding, policy, or approval first.

3. The wallet sends the token to the payee (a payment link, a profile, or a raw address).

4. Record — POST /api/v1/pay with the txHash. PayAgent reads the chain and checks amount, token, sender, and receiver.

5. If it matches, status is paid. That receipt is the settlement. Not a DID, not a PayAgent balance.

If the send works but the record call fails, retry the record only with the same txHash. Do not send the payment again.

With WDK you do not have to call those two HTTP steps yourself. After install, protocol.pay(...) does preflight ’ WDK transfer ’ record.

6. If the agent uses a WDK wallet

Install this on the agent machine. npm install only gets the library. It does not register the agent.

```
npm install @payagent/wdk-protocol @tetherto/wdk @tetherto/wdk-wallet-solana
```

For Ethereum or Base, use @tetherto/wdk-wallet-evm instead of the Solana module. The same seed on Solana and EVM makes two different addresses. Attach the one that matches the network.

Then, in your agent code:

- Open a WDK WalletAccount (seed stays on that machine).
- Read account.getAddress() — that is what you register.
- new PayAgentProtocol(account, { apiKey, agentId, networkId }).
- protocol.pay({ type: "address" | "link" | "profile", … }).

PayAgent never receives the seed. WDK only hands over getAddress and transfer. There is no paymaster. The wallet pays SOL or ETH for gas.

7. Worked example — two Solana agents

This folder (agents/wdk-test-agent) is a ready example. Two dummy agents hop 1 USDT on Solana mainnet. Same rules as above. Real SOL and official USDT — not a free testnet.

Agent A	wdk test agent - solana A
Agent B	wdk test agent - solana B
Token	USDT Es9vMFrzaCERmJfrF4H2FYD4KCoNkY11McCe8BenwNYB
What you see	A pays B, then B pays A. Solscan link + paid receipt.

From this folder (the template Tether runs):

- `npm install` then copy env.example to .env and add the merchant API key
- `npm run wallets` — two WDK Solana addresses
- `npm run setup` — register, attach wallets, USDT + Solana + no-approval policy
- `npm run check` — host, addresses, balances, preflight
- Confirm both names on Registered Agents (and Wallets ’ Agents)
- Fund both addresses with SOL. Fund at least one with official USDT.
- `npm run run` — waits for funds, then hops

8. If something does not work

• Agent missing in the portal — the API key belongs to another workspace, or register never ran.

• AGENT_WALLET_MISMATCH — the paying address is not the one you attached. Same seed on EVM vs Solana is a different address.

• Waiting for funds — no SOL (gas) or no official Solana USDT.

• Approval required — policy still wants a human yes. For a hands-off demo, requireApprovalAbove must be empty / null.

• Send worked, PayAgent not paid — retry record only. Do not send again.

• npm installed but nothing registered — expected. Install ` register.

9. Rules that do not change

• PayAgent never generates or stores a wallet seed.

• One agent is bound to the addresses you attached.

• Settlement is the on-chain txHash after PayAgent verifies it.

• The agent wallet pays gas. PayAgent is not a paymaster.

• No extra fee on this WDK path.

• Never commit .env, seeds, or API keys.

• This payment path is a working sidecar for partners to test. Do not call DIDs / VCs production settlement.

10. APIs used

• POST /api/v1/agents/register — create the agent name

• POST /api/v1/agents/wallet — bind the payer address

• PUT /api/v1/compliance/agents/:agentId/policy — spend rules

• GET /api/v1/pay/preflight — may this agent pay?

• POST /api/v1/pay — record the txHash after the wallet sends

Public docs: https://www.payagent.co/docs (WDK section: #wdk-protocol).

11. Share this, not the secrets

Share this PDF, the public wallet addresses, and the npm package name. Do not share .env, seeds, or the merchant API key. If someone else runs the example, they use their own key and their own seeds.

12. Repo commands (this folder only)

```
cd agents/wdk-test-agent
npm install
npm run wallets
npm run setup
npm run check
npm run run
```